Edge cases (search comes back empty) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `empty_search_practice.py`: an empty result list is the easy case — a real vector search almost never returns nothing; it returns the *closest* chunks, relevant or not. This exercise handles both. **If not:** the Build Task would answer from thin or empty context, confidently, which is exactly what its Constraints forbid.

Uses the same `practice/` setup as the Basic exercise (Doc08's files, `docs/` and `kb.py`). The only way to get a truly empty result is an empty store, so the code also builds one from an empty folder (Doc08's Build Task test did the same).

Basic Version

Approach 1 — the direct way

```
# empty_search_practice.py
from pathlib import Path
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from ingest import build_vector_store
from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
Path("empty_docs").mkdir(exist_ok=True)
empty_collection = build_vector_store("empty_docs", collection_name="empty")


class SearchState(TypedDict):
    task: str
    sources: list
    answer: str
```

```
def make_search_node(search_collection):
    def search_node(state):
        return {"sources": retrieve(search_collection, state["task"])}
    return search_node


def has_results(state):
    if not state["sources"]:
        return "no_grounding"
    return "reason"


def reason_node(state):
    pieces = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = "Context:\n" + context + "\n\nQuestion: " + state["task"]
    return {"answer": model.invoke(prompt).content}


def cant_answer_node(state):
    return {"answer": "I don't have grounding for this in my documents."}


def build_graph(search_collection):
    builder = StateGraph(SearchState)
    builder.add_node("search_node", make_search_node(search_collection))
    builder.add_node("reason_node", reason_node)
    builder.add_node("cant_answer_node", cant_answer_node)
    builder.add_edge(START, "search_node")
    builder.add_conditional_edges(
        "search_node", has_results,
        {"no_grounding": "cant_answer_node", "reason": "reason_node"},
    )
    builder.add_edge("reason_node", END)
    builder.add_edge("cant_answer_node", END)
    return builder.compile()


graph = build_graph(empty_collection)
result = graph.invoke({"task": "How hot should the water be when brewing?"})
print(result["answer"])
```

Expected output:

```
I don't have grounding for this in my documents.
```

`make_search_node(search_collection)` builds a node for whichever collection you pass in — a function that returns a function, the factory idea from Doc07. Here the graph searches the empty store, `has_results` sees an empty list, and routes to `cant_answer_node` instead of letting the model guess.

This catches a truly empty list. It doesn't catch the more common case: a few barely-relevant chunks.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, a `grounded` flag, and testing both paths

```
# empty_search_practice.py
from pathlib import Path
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from ingest import build_vector_store
from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
Path("empty_docs").mkdir(exist_ok=True)
empty_collection = build_vector_store("empty_docs", collection_name="empty")


class SearchState(TypedDict):
    task: str
    sources: list[dict]
    answer: str
    grounded: bool


def make_search_node(search_collection):
    def search_node(state: SearchState) -> dict:
        return {"sources": retrieve(search_collection, state["task"])}
    return search_node


def has_results(state: SearchState) -> str:
    if not state["sources"]:
        return "no_grounding"
    return "reason"


def reason_node(state: SearchState) -> dict:
    pieces: list[str] = []
```

```
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {"answer": model.invoke(prompt).content, "grounded": True}


def cant_answer_node(state: SearchState) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }


def build_graph(search_collection, router=has_results):
    builder = StateGraph(SearchState)
    builder.add_node("search_node", make_search_node(search_collection))
    builder.add_node("reason_node", reason_node)
    builder.add_node("cant_answer_node", cant_answer_node)
    builder.add_edge(START, "search_node")
    builder.add_conditional_edges(
        "search_node", router,
        {"no_grounding": "cant_answer_node", "reason": "reason_node"},
    )
    builder.add_edge("reason_node", END)
    builder.add_edge("cant_answer_node", END)
    return builder.compile()


if __name__ == "__main__":
    task = {"task": "How hot should the water be when brewing?"}
    in_scope = build_graph(collection).invoke(task)
    empty = build_graph(empty_collection).invoke(task)
    print("coffee docs -> grounded:", in_scope["grounded"])
    print("empty store -> grounded:", empty["grounded"])
```

Expected output:

```
coffee docs -> grounded: True
empty store -> grounded: False
```

Difference from Basic: full type hints, a `grounded` field every caller can check instead of parsing the answer text, and both paths tested. `build_graph()` also takes the router as a parameter, so Approach 2 can swap it.

Approach 2 — a distance threshold, not just an empty-list check

Story: an off-topic question still gets the closest chunks back — the list isn't empty, so Approach 1 sails on and the model "answers" from unrelated text. **If not:** a reviewer would see a confident answer with nothing behind it.

Add below Approach 1's code:

```
# empty_search_practice.py
from logging_setup import get_logger

MAX_DISTANCE = 1.2   # tune it: good matches below, off-topic ones above


def best_distance(sources: list[dict]) -> float:
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best


def has_relevant_results(state: SearchState) -> str:
    logger = get_logger("grounding")
    if not state["sources"]:
        logger.info(f"no_grounding reason=empty task={state['task']!r}")
        return "no_grounding"
    best = best_distance(state["sources"])
    if best > MAX_DISTANCE:
        logger.info(f"no_grounding reason=too_far best={best:.2f}")
        return "no_grounding"
    return "reason"


graph = build_graph(collection, router=has_relevant_results)
questions = ["How hot should the water be?", "How do I train for a marathon?"]
for question in questions:
    result = graph.invoke({"task": question})
    print(f"{question!r} -> grounded: {result['grounded']}")
```

Expected output (your distances will differ; tune `MAX_DISTANCE` on your own good and off-topic questions):

```
... grounding INFO no_grounding reason=too_far best=1.73
'How hot should the water be?' -> grounded: True
```

```
'How do I train for a marathon?' -> grounded: False
```

`score` is Doc08's **distance** — lower is closer — so the *best* match is the *smallest* score, and anything farther than `MAX_DISTANCE` doesn't count as grounding. (`1.2` is the same bar Project 3 uses.)

Approach 3 — a partial-grounding path

Story: sometimes search finds something relevant, but only for part of the question. Calling that "no grounding" throws away something real; calling it "grounded" risks answering the uncovered part anyway. **If not:** an answer that quietly ignored half the question would look the same as a fully-grounded one.

Add below Approach 2's code:

```
# empty_search_practice.py
GOOD_DISTANCE = 1.0


def grounding_level(state: SearchState) -> str:
    if not state["sources"]:
        return "no_grounding"
    best = best_distance(state["sources"])
    if best > MAX_DISTANCE:
        return "no_grounding"
    if best > GOOD_DISTANCE:
        return "partial"
    return "reason"


def partial_grounding_node(state: SearchState) -> dict:
    pieces: list[str] = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = (
        "The context may only partly answer the question. Answer only what "
        "the context supports, and say clearly what it doesn't cover.\n\n"
        f"Context:\n{context}\n\nQuestion: {state['task']}"
    )
    return {"answer": model.invoke(prompt).content, "grounded": "partial"}


def build_graph_with_partial(search_collection):
    builder = StateGraph(SearchState)
    builder.add_node("search_node", make_search_node(search_collection))
```

```
    builder.add_node("reason_node", reason_node)
    builder.add_node("partial_node", partial_grounding_node)
    builder.add_node("cant_answer_node", cant_answer_node)
    builder.add_edge(START, "search_node")
    builder.add_conditional_edges(
        "search_node", grounding_level,
        {
            "no_grounding": "cant_answer_node",
            "partial": "partial_node",
            "reason": "reason_node",
        },
    )
    builder.add_edge("reason_node", END)
    builder.add_edge("partial_node", END)
    builder.add_edge("cant_answer_node", END)
    return builder.compile()


graph = build_graph_with_partial(collection)
questions = [
    "How hot should the water be when brewing?",
    "What water temperature and which milk should I use for a latte?",
    "How do I train for a marathon?",
]
for question in questions:
    result = graph.invoke({"task": question})
    print(f"grounded={result['grounded']} -> {question}")
```

Expected output:

```
grounded=True -> How hot should the water be when brewing?
grounded=partial -> What water temperature and which milk should I use for
a latte?
grounded=False -> How do I train for a marathon?
```

(Wrapped to fit the page.) The latte question's best match is close enough to be useful (water temperature) but not close enough to cover everything (milk), so it gets an honest third answer, `"partial"`, and a prompt that says to name what's missing. (`grounded` now holds a `bool` or the text `"partial"` .)

Difference from Approach 1, and between Approaches 2/3: Approach 1 only asks "is the list empty?" — which misses the common case. Approach 2 adds a distance threshold and logs every fallback. Approach 3 adds an honest middle outcome instead of rounding a partial match up or down.

Which one should you actually write? Approach 1's empty check costs nothing — keep it. Add Approach 2's threshold once you've seen an off-topic question get "answered" (it will happen, with any embedding model). Add Approach 3 once you've seen a real answer ignore part of the question.